От запроса до покупки

Разбираем, как устроен магазин.
Затем собираем его по шагам.

Понять → Собрать → Проверить



↳ Начнём с одной покупки и проследим, что делает каждый участник.

Одна покупка — несколько действий

Аня уже вошла в аккаунт. Теперь она выбирает и покупает кроссовки.

01
Каталог
Найти North Runner

02
Размер
Выбрать 42

03
Корзина
Добавить 2 пары

04
Заказ
Подтвердить покупку

05
История
Увидеть свой заказ

Каждая стрелка — новое действие

Посмотреть товар, добавить позицию и оформить заказ — разные запросы.

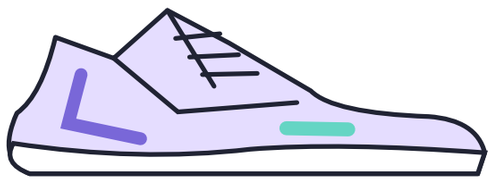
В корзине также есть City Walk × 1.
Итого: 2 × 7 990 + 5 990 = 21 970 ₽.

↳ Сервер нужен на каждом шаге, где мы читаем или меняем данные магазина.

Интерфейс показывает и отправляет

Кнопка связывает действие человека с запросом к серверу.

ЭКРАН ПОКУПАТЕЛЯ · ИЛЛЮСТРАЦИЯ



North Runner

7 990 ₽ за пару

42 ~~43~~

Выбрано: 2 пары

Добавить в корзину

1. **Показать данные**
   Название, цену и размеры интерфейс получил от API.
2. **Собрать выбор**
   Размер 42 соответствует записи с ID 101. Количество — 2.
3. **Отправить запрос**
   Сервер получит ID размера и количество, затем вернёт результат.

↳ Отключённая кнопка удобна пользователю. Правила покупки всё равно проверяет сервер.

Сервер проверяет правила магазина

Запрос — просьба выполнить действие. Сервер решает, допустимо ли оно.

Перед оформлением заказа

- ✓ Пользователь вошёл в аккаунт
- ✓ Корзина принадлежит ему
- ✓ Количество больше нуля
- ✓ На складе достаточно пар
- ✓ Сумма рассчитана по ценам из БД

Псевдокод · описывает идею — Учебный пример

```
пользователь = проверить_доступ()
корзина = найти_его_корзину()
проверить_количества(корзина)
проверить_остатки(корзина)
сумма = рассчитать_по_ценам_базы()
оформить_заказ(корзина, сумма)
```

↳ Пользователь сообщает свой выбор. Сервер проверяет его и рассчитывает результат.

База помнит то, что мы сохранили

Приложение можно перезапустить. Записанный заказ должен остаться.

ПРИЛОЖЕНИЕ

Обрабатывает запросы.
Временные значения живут в памяти.

перезапуск ↻ память процесса заново

После запуска

Приложение снова читает сохранённые данные из PostgreSQL.

СОХРАНЁННЫЙ ЗАКАЗ · ФРАГМЕНТ ТАБЛИЦЫ

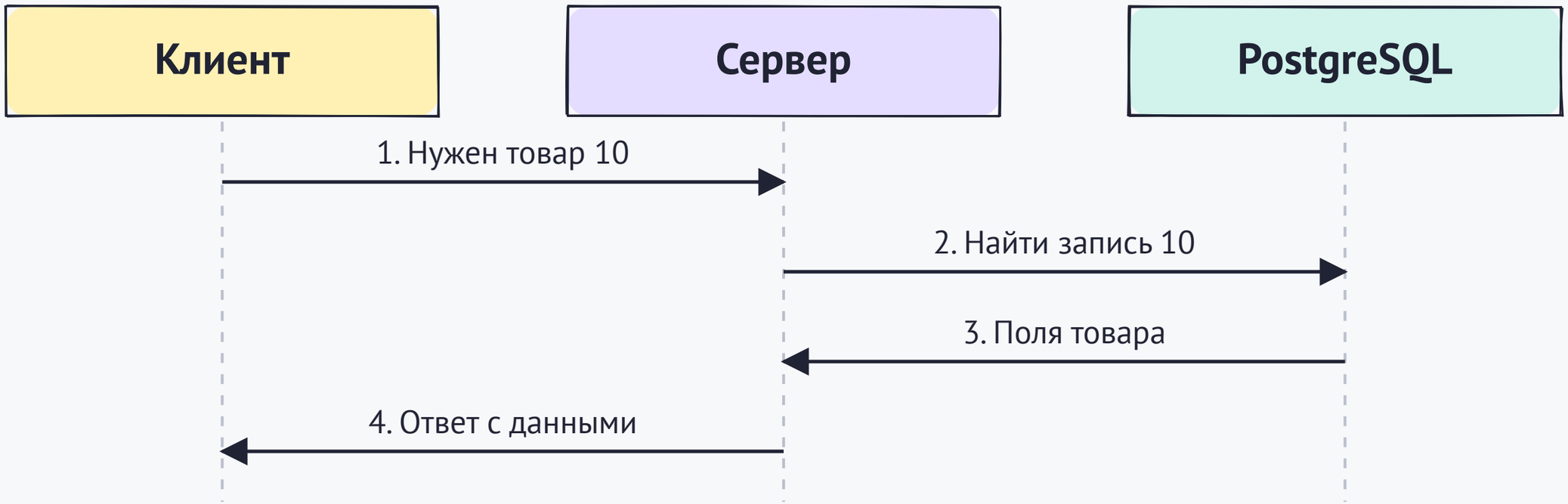
id	501
user_id	7 · Аня
total	21970.00
status	created

Каталог, пользователи, корзины и заказы хранятся в своих таблицах.

↳ Память приложения и постоянное хранилище решают разные задачи.

Клиент, сервер и база

Проследим чтение одной карточки товара.



↳ В этой архитектуре клиент обращается к API, а с базой работает сервер.

01 Что мы собираем

У каждой технологии своя работа

Пока достаточно понять место инструмента. Код разберём в следующих главах.

Запустить

Python + venv
Код и его окружение

Docker Compose
Запуск PostgreSQL

.env
Локальные настройки

Обработать запрос

FastAPI + Swagger
Маршруты и проверка API

Pydantic
Входные данные и ответы

passlib + python-jose
Пароли и токены доступа

Работать с данными

PostgreSQL
Хранение и запросы

SQLAlchemy
Работа с БД из Python

Alembic
Изменения схемы БД

↳ Инструменты складываются в одну систему: запуск → запрос → правила → данные.

Разбираем и сразу проверяем

Каждая тема приводит к небольшому действию, которое можно повторить.

01

Понять

Какую задачу решаем

02

Увидеть

Схема и данные

03

Прочитать

Код или псевдокод

04

Проверить

Что получилось

05

Применить

Шаг практики

Проверь себя перед второй главой

Где хранится заказ? Кто проверяет остатки?
Что отправляет клиент после нажатия кнопки?

↳ Следующая глава: читаем запрос и ответ так же уверенно, как сообщение человеку.

API — способ обратиться к программе

Мы заранее договариваемся, какой запрос отправить и какой ответ получить.

ПРОСЬБА КЛИЕНТА

«Покажи товары»

GET /products

Адрес и метод определяют операцию API.

JSON · ожидаемый фрагмент ответа — Учебный пример

```
[
  {
    "id": 10,
    "name": "North Runner",
    "price": "7990.00"
  }
]
```

↳ API задаёт понятный программам договор: вход, результат и возможные ошибки.

02 Как читать запросы и ответы

Разложим запрос на части

Аня добавляет две пары размера 42 в свою корзину.

HTTP · сокращённая запись запроса — Учебный пример

```
POST /cart/items HTTP/1.1
Host: localhost:8000
Content-Type: application/json
Authorization: Bearer <токен>

{
  "product_size_id": 101,
  "quantity": 2
}
```

1. **Метод: POST**
   Какую операцию просим выполнить.
2. **Путь: /cart/items**
   С какой частью магазина работаем.
3. **Заголовки**
   Адрес сервера, формат данных и доступ.
4. **Тело: JSON**
   Какой размер добавить и сколько пар.

↳ Метод + путь + заголовки + тело. Эти части читаем отдельно.

Как прочитать адрес API

URL показывает, куда обратиться и какие параметры передать.

```
http://localhost:8000/products?size=42
```

`http://`

Схема

Как обращаемся

`localhost`

Адрес узла

Этот компьютер

`:8000`

Порт

Вход в приложение

`/products`

Путь

Каталог

`?size=42`

Параметры

Фильтр размера

↳ На локальном стенде API и PostgreSQL слушают разные порты.

Путь выбирает конкретный товар

Число в адресе — идентификатор записи.

GET `/products/10`

ТАБЛИЦА PRODUCTS · ФРАГМЕНТ

id	name	price
10	North Runner	7990.00
20	City Walk	5990.00

/products/{id}

Шаблон маршрута. Вместо `{id}` клиент подставляет значение.

`/products/10` → North Runner
`/products/20` → City Walk

↳ ID товара, название товара и размер обуви — разные значения.

02 Как читать запросы и ответы

Параметры уточняют выборку

Меняйте фильтры: адрес и ожидаемый результат изменяются вместе.

Бренд **North · id 1** Размер **42** Учебная модель

GET `/products?brand_id=1&size=42`

Ожидаемый ответ: 200 **· 1 товар**

Товар	Цена	Размер · остаток
North Runner	7 990 ₽	42 · 3 пары

↳ После «?» идут параметры. Символ «&» соединяет несколько условий.

Тело передаёт данные изменения

Путь выбирает позицию корзины. JSON сообщает новое количество.

HTTP · сокращённая запись запроса — Учебный пример

```
PATCH /cart/items/55
Content-Type: application/json
Authorization: Bearer <токен>

{
  "quantity": 3
}
```

К КАКОЙ ЗАПИСИ

55 — ID позиции в корзине.
Он не равен ID товара или размера.

ЧТО ИЗМЕНИТЬ

quantity: 3 — установить три пары.
Было 2 → станет 3.

Сервер также проверит владельца позиции.

↳ Path: какую запись. Body: какие значения. Authorization: кто обращается.

02 Как читать запросы и ответы

JSON — данные с понятной структурой

Скобки, имена полей и типы помогают прочитать сообщение.

JSON · сокращённая карточка товара — Учебный пример

```
{
  "id": 10,
  "name": "North Runner",
  "sizes": [42, 43],
  "image_url": null
}
```

1 **{ } — объект**
Поля с именами: id, name, sizes, image_url.

2 **[ ] — массив**
Список значений. Здесь — два размера.

3 **"Текст", 10, null**
Строка, число и отсутствие значения.

4 **true / false — boolean**
Логическое значение. Например, is_admin в публичном профиле.

↳ Корректный JSON ещё не означает, что данные подходят нашему API.

Ответ тоже состоит из частей

Сначала узнаём исход запроса, затем читаем данные.

HTTP · сокращённый ожидаемый ответ — Учебный пример

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "name": "North Runner",
  "price": "7990.00"
}
```

1. **Статус: 200**
   Запрос выполнен успешно.
2. **Заголовки**
   Content-Type сообщает формат тела.
3. **Тело ответа**
   Поля карточки, которые получил клиент.

↳ Ответ нужно читать целиком: статус объясняет исход, тело — подробности.

02 Как читать запросы и ответы

Метод говорит, что мы хотим сделать

Один путь может поддерживать несколько разных операций.

Метод	Пример	Что просим
GET	`/products/10`	Прочитать карточку
POST	`/products`	Создать товар · администратор
PUT	`/products/10`	Заменить редактируемые поля карточки
PATCH	`/cart/items/55`	Изменить указанное поле: quantity
DELETE	`/cart/items/55`	Удалить позицию из своей корзины

↳ Для PUT передаём полный набор редактируемых полей. Для PATCH — выбранные изменения.

02 Как читать запросы и ответы

Адрес называет ресурс магазина

REST помогает организовать API вокруг данных и действий над ними.

`/products`	Каталог товаров
`/cart`	Своя корзина
`/brands`	Список брендов
`/orders`	Свои заказы

Коллекция

`GET /products`

Список карточек

Одна запись

`GET /products/10`

Карточка с ID 10

↳ Адрес выбирает ресурс, метод — действие, контракт — допустимые данные и результат.

Успех бывает разным

Статус помогает понять, что именно завершилось.

`200 OK`	`201 Created`	`204 No Content`
Получено	**Создано**	**Удалено**
`GET /products/10`	`POST /orders`	`DELETE /cart/items/55`
Карточка найдена. В теле — её данные.	Появился заказ. В теле — результат.	Позиция удалена. Тела ответа нет.
`{"id": 10, …}`	`{"id": 501, …}`	Пустое тело

↳ Это выбранные ответы нашего API. После 204 не пытаемся читать JSON.

Отказ тоже должен быть понятным

Одинаковая красная надпись скрывает разные причины.

401	Нет действительных данных входа	Корзина без токена
403	Недостаточно прав	Обычный пользователь создаёт товар
404	Ресурс не найден	Неизвестный ID товара
409	Конфликт с текущим состоянием	Заказываем 2 пары, осталась 1
422	Входные данные не прошли проверку	quantity = 0 при требовании больше 0

409 для нехватки остатка — выбранное соглашение магазина. Тело ответа объясняет причину.

↳ Сначала определяем причину отказа, затем решаем, что исправлять.

Иногда проблема на стороне сервера

Неожиданный сбой программы требует диагностики.

КЛИЕНТ ВИДИТ

500

Не удалось выполнить запрос.

Что выясняем

Какой запрос отправили?
Что ожидали?
Где возникло исключение?

Python traceback · условный пример — Учебный пример

```
Traceback (most recent call last):
  File "services/catalog.py", line 18
    return product.name
AttributeError:
  'NoneType' has no attribute 'name'
```

↳ Статус сообщает о сбое. Причину ищем в серверной диагностике и воспроизводимом запросе.

Теперь прочитай запрос сам

Назови действие, ресурс и каждое условие. Затем открой разбор.

```
GET /products?brand_id=1&size=42&max_price=8000
```

Читаем список товаров. Бренд — 1, размер — 42, цена — не выше 8 000 ₽. В наших данных подходит North Runner за 7 990 ₽.

Если совпадений нет: **200** и пустой список `[]`. Этот запрос не создаёт заказ и не списывает остаток.

Условия: бренд 1, размер 42, цена до 8 000 включительно. Ответ объясняет выбранный учебный контракт.

↳ Дальше научимся читать Python-код, который обрабатывает такие запросы.